10 USARE ASTRAZIONI E PATTERN

OBIETTIVI

- Imparare semplici principi per individuare pattern nei programmi.
- Discutere come l'astrazione si applica alla programmazione.
- Esaminare i tipi predefiniti disponibili in Python.
- Introdurre le classi per creare tipi e astrazioni personalizzate.
- Analizzare una serie di pattern pronti all'uso.

TROVARE PATTERN NEI PROGRAMMI

Nel **Capitolo 3** abbiamo parlato della ricerca di pattern nella decomposizione dei problemi. Quando un problema viene scomposto in sotto-problemi, emergono elementi simili che possono essere raggruppati o generalizzati.

Questo processo continua anche durante la scrittura del codice: più programmi, più pattern riconosci e puoi sfruttarli per semplificare il codice.

Un principio chiave della programmazione è:

✅ **Non ripetere te stesso** (Don't Repeat Yourself - DRY)
✅ **Trova cosa varia e incapsulalo**

💡 **Esempio: Disegnare una faccia con tkinter**

Nel Capitolo 9, avevamo questa funzione:

python

CopiaModifica

```
def render_neutral_face():
    core_draw.draw_oval(50, 50, 120, 120)  # Viso
    core_draw.draw_oval(60, 60, 80, 80)    # Occhio sinistro
    core_draw.draw_oval(90, 60, 110, 80)   # Occhio destro
    core_draw.draw_line(70, 110, 100, 110) # Bocca
```

🔴 **Problema**: ripetizione di codice.

✅ **Soluzione**: usare un ciclo per evitare la ripetizione.

python

CopiaModifica

```
def render_neutral_face():
  cerchi = [(50, 50, 120, 120), (60, 60, 80, 80), (90, 60, 110, 80)]
  for d in cerchi:
    core_draw.draw_oval(*d)
  core_draw.draw_line(70, 110, 100, 110)
```

Ora la funzione è più chiara e più facile da modificare.

ASTRAZIONI NELLA PROGRAMMAZIONE

Un'**astrazione** permette di nascondere i dettagli inutili e lavorare con concetti più semplici.

Esempio: una variabile come x = 42 è un'astrazione. In realtà, Python gestisce molti dettagli nascosti (dove il valore è salvato in memoria, quanto spazio occupa, ecc.).

Esempi di astrazioni in Python

✅ **Variabili** → Nascondono la gestione della memoria.
✅ **Liste, tuple, dizionari** → Nascondono dettagli interni di gestione dei dati.
✅ **Cicli for e while** → Nascondono le operazioni ripetitive di basso livello.

TIPI PREDEFINITI IN PYTHON

Python fornisce tipi di dati già pronti per memorizzare informazioni.

Tipo	Esempio	Descrizione
int	x = 42	Numero intero.
float	y = 3.14	Numero con decimali.
bool	vero = True	Valore booleano (True/False).
list	numeri = [1, 2, 3]	Sequenza modificabile.
tuple	coordinate = (10, 20)	Sequenza immutabile.
set	pari = {2, 4, 6}	Collezione di valori unici.
dict	età = {'Anna': 25, 'Luca': 30}	Dizionario (mappa chiave-valore).

💡 **Attenzione ai numeri decimali**
I numeri float possono essere imprecisi. Ad esempio, in Python:

python

CopiaModifica

```
print(1.1 + 2.2)  # Output: 3.3000000000000003
```

Per calcoli precisi, usa il modulo decimal.

CREARE TIPI PERSONALIZZATI

Se i tipi predefiniti non bastano, puoi creare il tuo tipo con le **classi**.

Esempio: una classe Car per rappresentare automobili.

python

CopiaModifica

```
class Car:
    category = 'B'

fiat = Car()
print(fiat.category)  # Output: B
```

💡 **Ma serve personalizzazione!** Ogni auto dovrebbe avere un modello e un chilometraggio.

python

CopiaModifica

```
class Car:
    category = 'B'

    def __init__(self, model, mileage):
        self.model = model
        self.mileage = mileage

fiat = Car('Fiat Panda', 12000)
print(fiat.model)  # Output: Fiat Panda
```

print(fiat.mileage) # Output: 12000

Ora possiamo creare più oggetti Car con dati diversi.

EREDITARIETÀ

L'**ereditarietà** permette di creare classi più generali e riutilizzarle.

Esempio: invece di definire Car, Van e Motorcycle separatamente, creiamo una classe Vehicle da cui ereditano.

python

CopiaModifica

```
class Vehicle:
    def __init__(self, model, mileage):
        self.model = model
        self.mileage = mileage

    def update_mileage(self, new_mileage):
        if new_mileage < self.mileage:
            print("Errore: il nuovo valore deve essere maggiore!")
        else:
            self.mileage = new_mileage

class Car(Vehicle):
    category = 'B'

class Van(Vehicle):
    category = 'C'

class Motorcycle(Vehicle):
    category = 'A'
```

Ora Car, Van e Motorcycle ereditano **automaticamente** metodi e attributi da Vehicle.

POLIMORFISMO

Il **polimorfismo** permette di usare classi diverse nello stesso modo.

Esempio:

python

CopiaModifica

```
vehicles = [Car('Fiat', 12000), Van('Ford', 50000), Motorcycle('Honda', 3000)]


for v in vehicles:
    print(v.model)  # Funziona per tutti i tipi di veicoli!
```

PATTERN DI PROGRAMMAZIONE

Ci sono pattern ricorrenti in molti programmi. Alcuni esempi:

✅ **Pattern Facade** → Nasconde la complessità dietro un'interfaccia semplice.
✅ **Pattern Factory** → Crea oggetti in modo flessibile.
✅ **Pattern Method Chaining** → Permette di concatenare metodi.

Esempio di **Method Chaining**:

python

CopiaModifica

```
x = "ciao"

y = x.upper().replace("A", "E")

print(y)  # Output: "CIEO"
```

RIASSUNTO

✓ **Non ripetere il codice**: usa funzioni, cicli e classi.
✓ **Usa le astrazioni**: tipi, classi e pattern semplificano il codice.
✓ **Ereditarietà e polimorfismo** migliorano la struttura del codice.
✓ **Usa pattern di programmazione** per evitare di reinventare la ruota.